

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

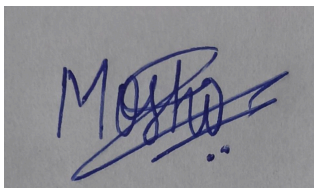
Annexure A

Constraint Based Problem Solving

Code of Conduct:

*I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Moshika', is written over a light gray rectangular background.

Annexure A

Short Answer Test

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

ANSWER:

A patient health-record dataset may contain missing values, invalid numerical values, inconsistent formats, and outliers. Using Pandas and NumPy, a cleaning pipeline can be designed to enforce predefined constraints while preserving valid records.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Age": [25, 34, 45, 67, 150, 29, 52, None, 78, -5,
           41, 63, 37, 89, 55, None, 120, 32, 48, 70],
    "BP": [120, 130, 140, 150, 300, 110, 125, None, 180, 30,
          135, 145, 118, 200, 128, None, 160, 115, 122, 170],
    "Gender": ["M", "Male", "F", "female", "M", " f ", "MALE",
              None, "Female", "F", "m", "Male", "female", "F",
              "M", None, "male", "F", "female", "M"]
})

df.loc[(df["Age"] < 0) | (df["Age"] > 120), "Age"] = np.nan
df.loc[(df["BP"] < 40) | (df["BP"] > 250), "BP"] = np.nan

df["Age"] = df["Age"].fillna(df["Age"].median())
df["BP"] = df["BP"].fillna(df["BP"].median())

df["Gender"] = df["Gender"].str.strip().str.lower()
df["Gender"] = df["Gender"].replace({"m": "male", "f": "female"})
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])

print(df)
print("\nMissing Values:", df.isnull().sum().sum())
print("Valid Ages:", df["Age"].between(0, 120).all())
print("Valid BP:", df["BP"].between(40, 250).all())
```

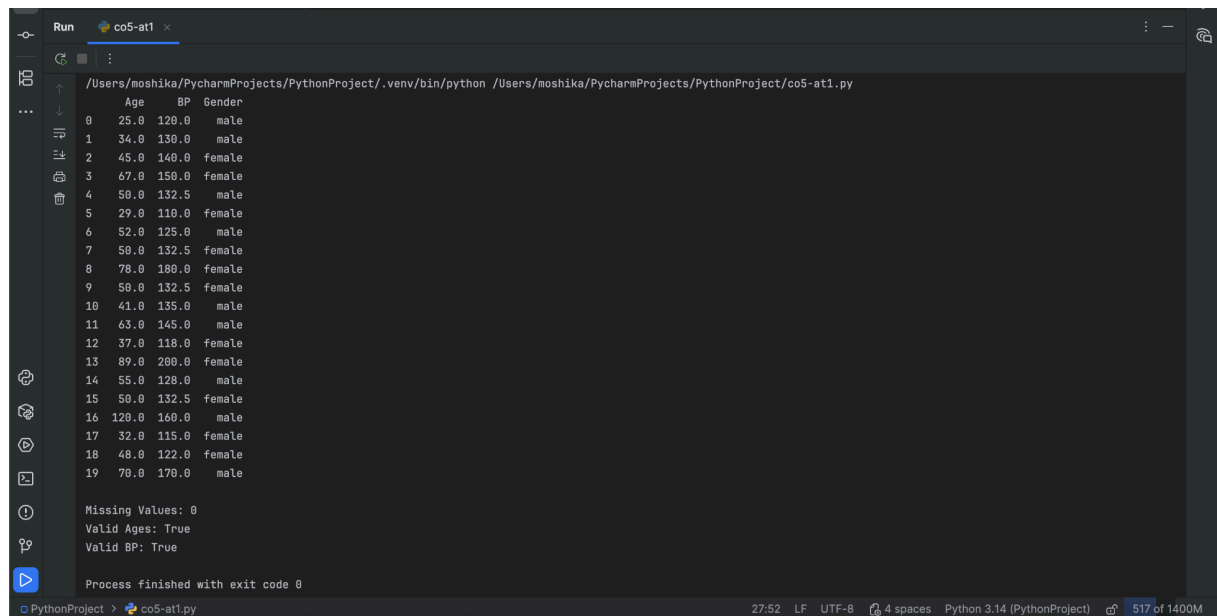
Indexing using `loc[ ]` helps identify and correct invalid values. **Filtering** can locate records outside valid medical ranges. **Aggregation functions** such as `median()`, `mode()`, `min()`, and `max()` help fill missing values and validate the cleaned dataset. Outliers can be detected using IQR or domain-specific medical limits.

Finally, constraints can be verified using:

```
print(df.isnull().sum().sum())
print(df["Age"].between(0, 120).all())
```

Thus, the pipeline removes missing values, ensures valid numerical ranges, standardizes categories, and retains as many valid patient records as possible.

OUTPUT :



The screenshot shows a Jupyter Notebook interface with a table of patient data and validation results. The table has columns for Age, BP, and Gender. The data is as follows:

	Age	BP	Gender
0	25.0	120.0	male
1	34.0	130.0	male
2	45.0	140.0	female
3	67.0	150.0	female
4	50.0	132.5	male
5	29.0	110.0	female
6	52.0	125.0	male
7	50.0	132.5	female
8	78.0	180.0	female
9	50.0	132.5	female
10	41.0	135.0	male
11	63.0	145.0	male
12	37.0	118.0	female
13	89.0	200.0	female
14	55.0	128.0	male
15	50.0	132.5	female
16	120.0	160.0	male
17	32.0	115.0	female
18	48.0	122.0	female
19	70.0	170.0	male

Below the table, the output shows the results of the validation checks:

```
Missing Values: 0
Valid Ages: True
Valid BP: True
```

The process finished with exit code 0.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

ANSWER:

For a massive numerical dataset, memory can be controlled by using suitable data types, slicing, in-place operations, and efficient aggregation.

```
import numpy as np
```

```

data = np.arange(1, 1000001, dtype=np.float32)

print("Memory:", data.nbytes, "bytes")

# Slicing without copying
sample = data[:100]

# In-place operation
data *= 2
data += 10

# Statistics
mean = data.mean()
std = data.std()
minimum = data.min()
maximum = data.max()

# Random sampling
indices = np.random.choice(len(data), 1000, replace=False)
random_sample = data[indices]

print("Mean:", mean)
print("Standard Deviation:", std)
print("Minimum:", minimum)
print("Maximum:", maximum)
print("Sample Size:", len(random_sample))

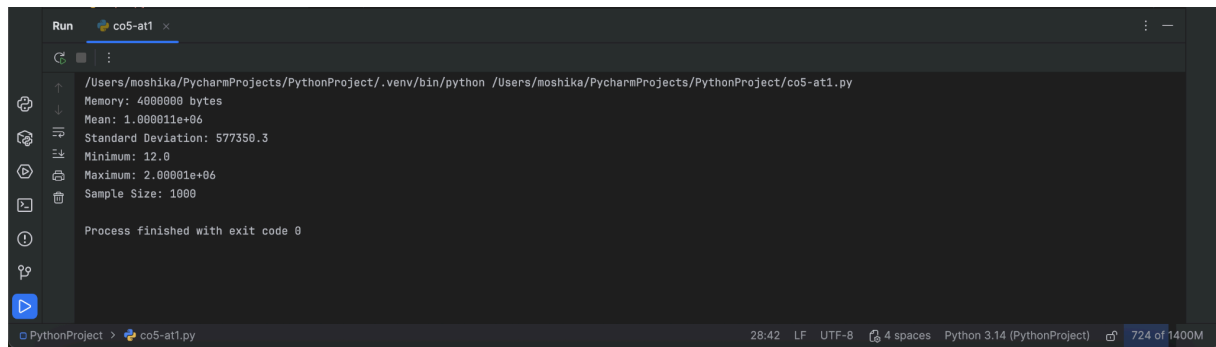
```

Using `float32` instead of `float64` reduces memory consumption when the required precision allows it. NumPy slicing usually creates a **view**, avoiding an unnecessary copy of the data. In-place operations such as `*=`, `+=`, and `/=` also reduce temporary memory allocation.

Aggregation should be performed directly on the required array rather than creating intermediate arrays. Boolean filtering should also be used carefully because it can create a new array.

If the dataset is too large, it can be processed in **chunks** instead of loading everything at once. For debugging, `data.nbytes`, `data.shape`, and `data.dtype` can be checked to identify excessive memory usage. Performance bottlenecks can be reduced by replacing Python loops with vectorized NumPy operations.

OUTPUT :



```
Run co5-at1 x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co5-at1.py
Memory: 4080000 bytes
Mean: 1.000011e+06
Standard Deviation: 577350.3
Minimum: 12.0
Maximum: 2.00001e+06
Sample Size: 1000
Process finished with exit code 0
```

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

ANSWER:

Two DataFrames containing transaction and demographic information can be combined using **CustomerID** as the common key. Before merging, duplicate and missing IDs should be checked.

```
import pandas as pd
```

```
transactions = pd.DataFrame({
    "CustomerID": list(range(101, 121)),
    "Amount": [500, 750, 300, 900, 450, 620, 800, 350, 700, 550,
               1000, 400, 650, 850, 250, 720, 480, 560, 930, 610]
})
```

```
customers = pd.DataFrame({
    "CustomerID": list(range(101, 121)),
    "Name": ["Asha", "Ravi", "John", "Meena", "Kiran",
             "Arun", "Priya", "Rahul", "Divya", "Anu",
             "Vijay", "Neha", "Rohit", "Sneha", "Ajay",
             "Pooja", "Manoj", "Swathi", "Karthik", "Nisha"],
    "Age": [25, 30, 28, 35, 40, 27, 32, 29, 31, 26,
            45, 38, 34, 29, 41, 36, 33, 28, 39, 30]
})
```

```
customers = customers.drop_duplicates("CustomerID")
```

```
missing = transactions[
    ~transactions["CustomerID"].isin(customers["CustomerID"])]
```

```
]
```

```
merged = transactions.merge(  
    customers,  
    on="CustomerID",  
    how="left",  
    validate="many_to_one"  
)
```

```
merged = merged.sort_values("CustomerID").reset_index(drop=True)
```

```
print("Missing IDs:")  
print(missing)
```

```
print("\nCombined Data:")  
print(merged)
```

```
print("\nRecords per Customer:")  
print(merged.groupby("CustomerID").size())
```

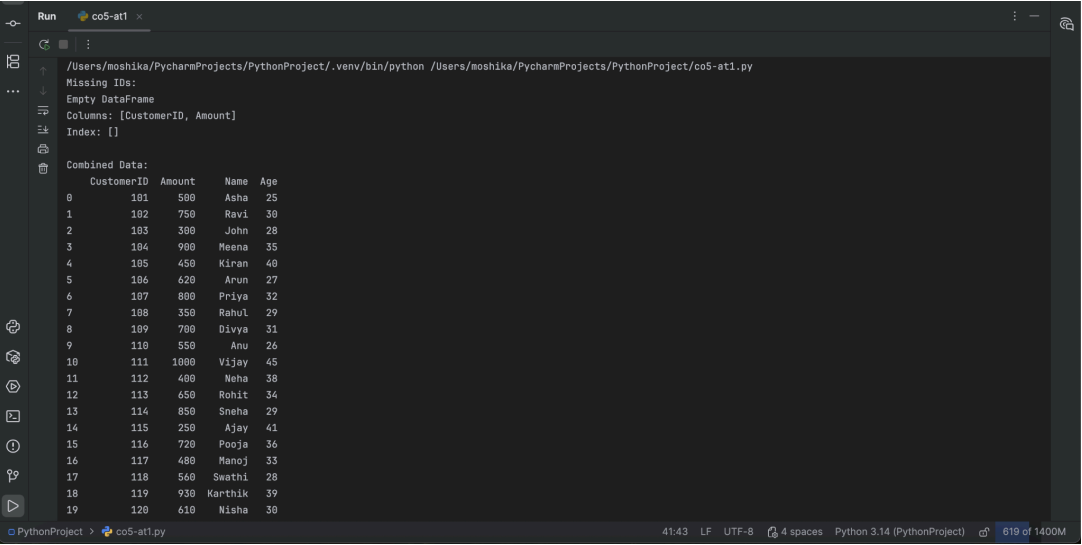
A **left join** is used because it preserves all valid transaction records even if demographic information is missing. The `validate="many_to_one"` option helps ensure that one customer does not have multiple demographic records unexpectedly.

Filtering with `isin()` identifies unmatched customer IDs, while `groupby()` can be used to detect unexpected duplicates:

```
print(merged.groupby("CustomerID").size())
```

Finally, sorting and resetting the index provides a consistent index in the resulting DataFrame. Mismatched entries can be investigated before deciding whether they should be corrected or removed.

OUTPUT :



The screenshot shows a Jupyter Notebook interface with the following output:

```
Missing IDs:  
Empty DataFrame  
Columns: [CustomerID, Amount]  
Index: []
```

Combined Data:

	CustomerID	Amount	Name	Age
0	101	500	Asha	25
1	102	750	Ravi	30
2	103	300	John	28
3	104	900	Meena	35
4	105	450	Kiran	40
5	106	620	Arun	27
6	107	800	Priya	32
7	108	350	Rahul	29
8	109	700	Divya	31
9	110	550	Anu	26
10	111	1000	Vijay	45
11	112	400	Neha	38
12	113	650	Rohit	34
13	114	850	Sneha	29
14	115	250	Ajay	41
15	116	720	Pooja	36
16	117	480	Manoj	33
17	118	560	Swathi	28
18	119	930	Karthik	39
19	120	610	Nisha	30

The bottom status bar of the Jupyter Notebook shows: PythonProject > co5-at1.py 41:43 LF UTF-8 4 spaces Python 3.14 (PythonProject) 619 of 1400M

```
...
Records per Customer:
CustomerID
101 1
102 1
103 1
104 1
105 1
106 1
107 1
108 1
109 1
110 1
111 1
112 1
113 1
114 1
115 1
116 1
117 1
118 1
119 1
120 1
dtype: int64

Process finished with exit code 0

PythonProject > co5-at1.py 41:43 LF UTF-8 4 spaces Python 3.14 (PythonProject) 740 of 1400M
```

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

ANSWER:

Mean, median, and variance can be calculated efficiently using NumPy or Pandas aggregation functions. Before calculation, irrelevant or invalid values should be filtered.

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "Value": np.arange(10, 10010, 10)
})

x = df["Value"].dropna()

mean = np.mean(x)
median = np.median(x)
variance = np.var(x)

print("Number of Values:", len(x))
print("Mean:", mean)
print("Median:", median)
print("Variance:", variance)
print("Minimum:", np.min(x))
print("Maximum:", np.max(x))
```

Filtering removes missing or invalid observations before performing calculations. **Indexing** can select only the required columns or rows, reducing unnecessary computation.

NumPy's vectorized aggregation functions are faster than manually calculating values using loops. To avoid precision loss, `float64` can be used when high numerical accuracy is required.

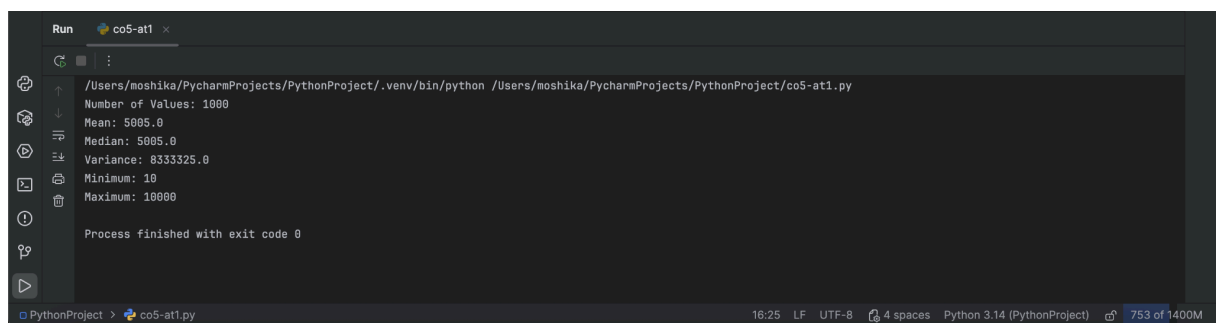
Variance should also be calculated with the correct definition. For example:

```
np.var(x, ddof=0) # Population variance
np.var(x, ddof=1) # Sample variance
```

To debug incorrect results, check the number of observations, missing values, data type, minimum and maximum values, and the `ddof` parameter. The results can also be compared with Pandas functions such as `df["Value"].mean()` and `df["Value"].var()`.

Thus, efficient filtering, indexing, and aggregation provide accurate statistical results while avoiding unnecessary computations.

OUTPUT :



```
Run co5-at1 x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co5-at1.py
Number of Values: 1000
Mean: 5005.0
Median: 5005.0
Variance: 8333325.0
Minimum: 10
Maximum: 10000
Process finished with exit code 0
PythonProject > co5-at1.py 16:25 LF UTF-8 4 spaces Python 3.14 (PythonProject) 753 of 1400M
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand